

Pins need grid access

A detailed route starts and ends at grid access points, integer cells the router can reach

The idea

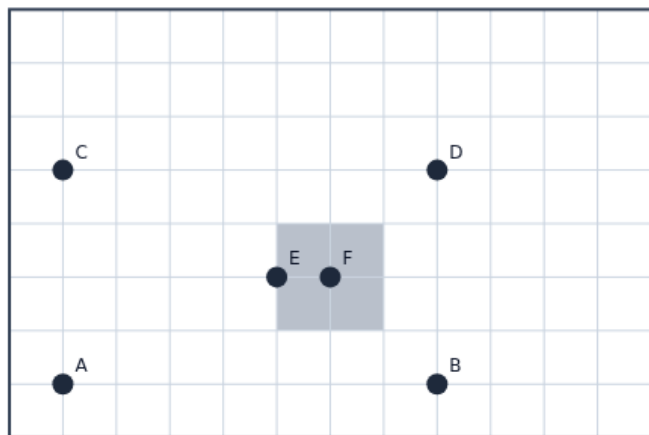
- `Pin_grid` of `x y` returns column `gx` equals round of `x` clamped zero to `nx` minus one
- Cell A at one comma one maps to one comma one
- Cell E at five comma three sits near the blockage at five comma two

Pin → grid

PIN ACCESS

Pin → grid

Step 1 / 5 · map · module01-03-pin-access



Explanation

Cell centers round to grid indices.

- Same spread as global

Clamp on-chip

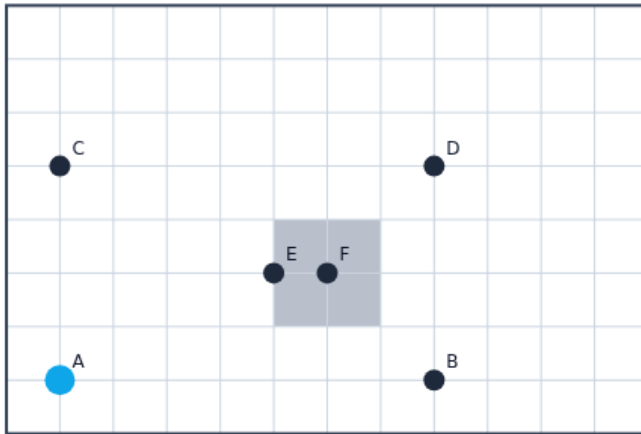
Pin → grid

A → (1,1)

PIN ACCESS

A → (1,1)

Step 2 / 5 · a11 · module01-03-pin-access



Explanation

Spread A lands on grid (1,1).

- Terminals move with cells

A 1,1

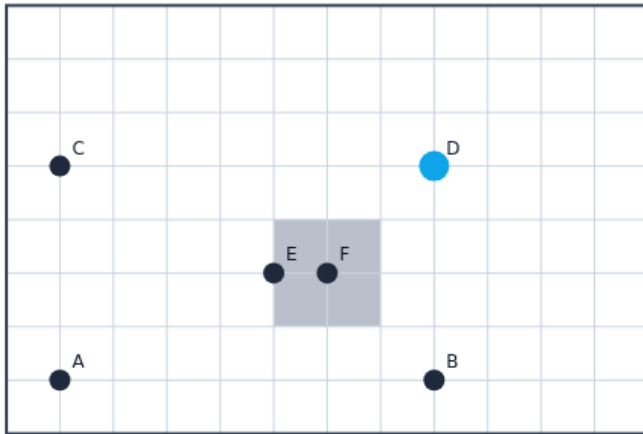
A → (1,1)

D → (8,5)

PIN ACCESS

D → (8,5)

Step 3 / 5 · d85 · module01-03-pin-access



Explanation

D at (8,5) top row.

- Top row y=5

D 8,5

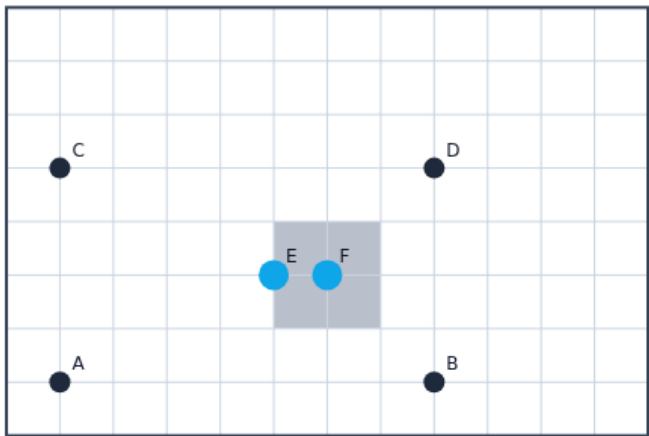
D → (8,5)

Blocked pins

PIN ACCESS

Blocked pins

Step 4 / 5 · nudge · module01-03-pin-access



Explanation

E/F on blockage nudge to nearest free neighbor.

- Access point
- Toy only

Pin access

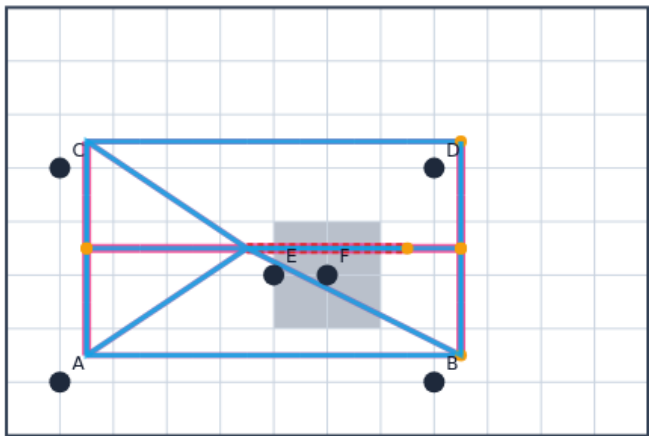
Blocked pins

Routes from pins

PIN ACCESS

Routes from pins

Step 5 / 5 · route · module01-03-pin-access



Explanation

Sequential routes start/end at terminal grid points.

- Not bbox

Track usage

Routes from pins

Browser lab track

- Open **pin-access**
- Hover each cell and read its access grid point
- Move a pin across the blockage boundary and watch the access index shift
- Confirm all six cells on tiny_dr match your Track A printout

Implement track

- Implement or call `terminals(positions, data)` in `common/drutil.py`
- Assert A through F on spread placement
- Print access points before any routing lab

Pitfalls

- Using GCell floor divide from global routing instead of per-grid rounding
- Off-by-one at the chip edge without clamp
- Ignoring blockages when assigning access

Your turn

- Complete access points for all six cells
- Next: Lee maze routing around the central blockage